У даній лабі необхідно було . . .

```
class IMDBDataset(Dataset):
    def __init__(self, split):
        self.data = dataset[split]

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        text = self.data[idx]["text"]
        label = self.data[idx]["label"]
        encoding = tokenizer(text, truncation=True, padding="max_length",
max_length=128, return_tensors="pt")
        return encoding["input_ids"].squeeze(0),
encoding["attention_mask"].squeeze(0), torch.tensor(label,
dtype=torch.long)
```

Значить ось прописаний класс `IMDBDataset`. Клас створений щоб підготувати текстові дані (IMDB) у форматі з яким може працювати PyTorch-модель.
Він обробляє текст (**ну токенізує його, відповідно токен це одиниця тексту, яку модель обробляє, це може бути слово, там частина слова, якийсь знак. Ну тобто текст у нас розбивається у токени, токени в числа, числа у вектори, які вже подаються у модель**).
В цілому клас потрібен, щоб **передавати IMDB-дані в DataLoader**, який буде формувати батчі для навчання або тестування.
(**ну власне батч це пакет або ж група прикладів з датасету, які подаються в модель одночасно під час навчання. В цілому батчі необхідні для покращення ефективності**.)

```
class LSTMWithAttention(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(LSTMWithAttention, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
bidirectional=True)
        self.attn = nn.Linear(hidden_dim * 4, hidden_dim)
        self.v = nn.Parameter(torch.rand(hidden_dim))
        self.fc = nn.Linear(hidden_dim * 2, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        lstm_out, _ = self.lstm(embedded)
        hidden = lstm_out[:, -1, :].unsqueeze(1).repeat(1,
lstm_out.shape[1], 1)
        concat = torch.cat((hidden, lstm_out), dim=2)
        energy = torch.tanh(self.attn(concat)).matmul(self.v)
        attn_weights = torch.softmax(energy, dim=1)
        context_vector = torch.sum(attn_weights.unsqueeze(2) * lstm_out,
dim=1)
        return self.fc(context_vector), attn_weights
```

Далі у мене прописаний класс LSTMwithATTENTION. Його основна суть це :

- Перетворення токени у вектори
- Обробка їх за допомогою **двонаправленої** **LSTM** **(** LSTM — це покращена версія **рекурентної нейронної мережі** (RNN), яка вирішує проблему затухаючих градієнтів та дозволяє працювати з довгими послідовностями. Складається ця мережа з
  ✅ **Вхідний стан** (input gate) – стан який контролює, яка інформація додається до комірки пам’яті.
  ✅ **Стан пам’яті** (cell state) – зберігає довготривалу інформацію.
  ✅ **Забудькуватий елемент** (forget gate) – вирішує, яку частину інформації потрібно видалити.
  ✅ **Вихідний елемент** (output gate) – визначає, яку частину інформації передати далі.

- Використовування **механізму уваги (Attention)** (тобто методу, який дозволяє моделі **визначити, які частини вхідної інформації найважливіші** для виконання завдання) ну власне щоб зважити, які токени важливіші,

📌 **Основні етапи механізму уваги:**

1️⃣ **Обчислення балів (scores)**

- Для кожного стану обчислюється "важливість" для поточного виходу:

2️⃣ **Застосування softmax**

- Нормалізуємо значення для отримання ваг (attention weights):

3️⃣ **Обчислення контекстного вектора**

- Контекстний вектор — це зважена сума всіх прихованих станів:

4️⃣ **Використання контекстного вектора**

- Він передається у вихідний шар для генерації остаточного результату.

- Формування **контекстного вектору** і передача його у класифікатор.

def forward
як дані проходять через модель

```
vocab_size = tokenizer.vocab_size - кількість унікальних токенів у
словнику токенізатора.
embed_dim = 128  -  розмірність
hidden_dim = 256 - кількість нейронів у прихованому шарі LSTM.
output_dim = 2 - кількість класів

lstm_model = LSTMWithAttention(vocab_size, embed_dim, hidden_dim,
output_dim).to(device)
lstm_optimizer = optim.Adam(lstm_model.parameters(), lr=2e-4)

def compute_metrics_lstm(model, loader):
```

Ініціалізуємо модель LSTM
Налаштовуємо оптимізатор для її навчання.
Функція для перевірки, наскільки добре модель працює:

- Тобто ми проходимо по всьому набору іеруємось по всіх батчах даних, обраховуємо та повертаємо втрати, точність та F1 (**якість моделі**, яка **враховує і точність (precision), і повноту (recall)**.)

```
def compute_metrics_bert(model, loader):
```

ну і аналогічна функція але вже для BERT моделі. Власне **BERT це модель** яка побудована на **енкодерах** трансформера (**трансформери** це така архітектура нейронних мереж для обробки тексту, вона використовує **self-attention** для роботи з довгими послідовностями. Ну тобто відбувається паралельна обробка всього тексту, використовуючи self-attention)
Що до функції, то вона абсолютно аналогочна до попередньої, тобто ми просто робимо обрахунки точності та втрат, та записуємо їх у списпи для подальшої побудови графіків

```
def train_lstm(model, train_loader, test_loader, optimizer, epochs=5):
```
далі я прописав функцію для train(у) та test(у) моделі LSTM. Тобто у функції виконуєтся навчання моделі на батчах даних. Після кожної епохи обчислюються метрики (loss, accuracy, F1) . Ну і відповідно прописав вивід результатів та їх збереження для того щоб побудувати графіки.

```
def fine_tune_bert(model, train_loader, test_loader, optimizer, epochs=5):
```
Далі йде функція `fine_tune_bert,` вона використовується для донавчання попередньо натренованої DistilBERT моделі у моєму випадку це класифікація відгуків з набору даних IMDB
Тобто модель проходить кілька епох навчання (у мене це 5). Для кожної епохи модель:

- опрацьовує тренувальні дані батчами (групами прикладів),
- обчислює втрату (loss),
- виконує зворотне поширення помилки і оновлює ваги,
- ну і відповідно після завершення епохи — оцінює свою якість на train + test наборах, обчислюючи:
    - середню втрату (loss),
    - точність (accuracy),
    - F1-метрику.

```
def visualize_attention_weights(input_ids, attention_weights, tokenizer):
```
Далі у мене функція візуалізації механізму уваги в LSTM-моделі, якщо коротко то вона бере один приклад з тестового набору, пропускає його через LSTM-модель з увагою, отримує ваги уваги та візуалізує їх у вигляді діаграми, щоб показати, **на які слова модель звертала найбільшу увагу** при прийнятті рішення . ну тобто показує **як саме модель "розуміє" текст**.

Батч — це **пакет (група)** прикладів з датасету, які подаються в модель **одночасно** під час навчання або передбачення. Батчі необхідні для покращення ефективності.

Токен — це **одиниця тексту**, яку модель обробляє, це може бути слово, там частина слова, якийсь знак. Ну тобто **текст** у нас розбивається у **токени**, **токени** в **числа**, **числа** у **вектори,** які вже подаються у модель

LSTM-модель (Long Short-Term Memory) — це різновид рекурентної нейронної мережі (RNN) ну її покращений варіант, добре працює з послідовними даними, наприклад, текстами або часовими рядами. Вона краще за звичайну RNN пам’ятає важливу інформацію з попередніх елементів і забуває непотрібну завдяки механізмам керування (input, forget, output, та cell gates)

BERT– ну це модель, яка вміє розуміти текст завдяки тому, що аналізує його одночасно зліва направо і справа наліво. Вона створена на основі архітектури трансформерів і попередньо навчена на великій кількості текстів

Трансформери – архітектура нейронних мереж, яка працює з послідовностями. Особливість їх у тому, що замість обробки слів по черзі, як у звичайних RNN, трансформери обробляють **всі слова одночасно**, використовуючи механізм **уваги (attention)**.

F1 - метрика – це показник якості класифікаційної моделі, вона одночасно враховує і точність (precision), і повноту (recall). Ну тобто показує, наскільки добре модель вгадує правильні відповіді, і не пропускає важливі випадки.

Механізм уваги — це спосіб у нейронних мережах "зосереджуватись" на важливих частинах вхідної інформації. Наприклад, коли модель обробляє речення, механізм уваги допомагає їй визначити, на які саме слова потрібно звернути більше уваги.